# STL Containers

Containers hold data. Memory layout, operation costs and the choice between vector, array, deque and list are the parts that matter most. Assumes Pointers and References.

## The big picture




Algorithms never touch containers, only iterator ranges.

One sort works on any container that offers the right iterators.

| PART | WHAT IT IS | EXAMPLES |
|---|---|---|
| **Containers** | classes that own and organise a collection of elements | `vector`, `map`, `set`, `deque` |
| **Iterators** | generalised pointers that walk a container | `v.begin()`, `m.find(k)` |
| **Algorithms** | generic functions that operate on iterator *ranges* | `sort`, `find`, `accumulate` |

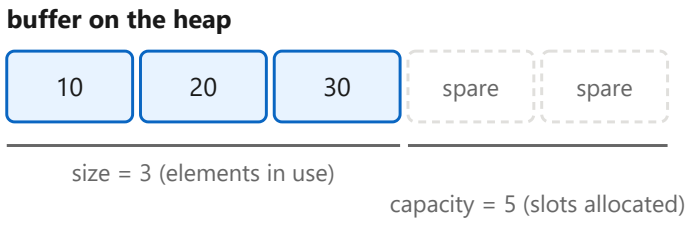
Algorithms see only `[first, last)`, so one `std::sort` serves a vector, array or deque. That separation is the design.

```
#include <vector>       // std::vector
#include <map>          // std::map, std::multimap
#include <unordered_map>// std::unordered_map
#include <set>          // std::set
#include <algorithm>    // std::sort, std::find, std::transform ...
#include <numeric>      // std::accumulate, std::iota
#include <string>       // std::string
```

**DEFAULT CHOICE** `std::vector`. Contiguous memory is so cache-friendly it often beats "better" containers.

---

## `vector`: the workhorse




**push_back, buffer full:**
allocate bigger, move all, free old
= amortised O(1), but every iterator and pointer into it is invalidated

reserve() grows capacity only; resize() changes size.

A `vector` is one contiguous heap array with a **size** and a **capacity**. When full, `push_back` allocates a bigger buffer, moves everything and frees the old one: occasional O(n), **amortised O(1)**.

```
std::vector<int> v;
// capacity 100, size still 0: no reallocation for the next 100 push_backs
v.reserve(100);
v.push_back(1);          // copy/move an existing object in
v.emplace_back(2);       // construct the element in place: no temporary
// size becomes 10; new elements are value-initialised (0)
v.resize(10);

// no bounds check: out-of-range is undefined behavior
v[3];
v.at(3);                 // bounds-checked: throws std::out_of_range
v.front(); v.back(); v.data();   // data() is the raw int* for C APIs
v.size(); v.capacity(); v.empty();
```

**RESERVE VS RESIZE** `reserve(n)` only allocates; `resize(n)` creates elements. `v[5] = x` after only `reserve(10)` is UB.

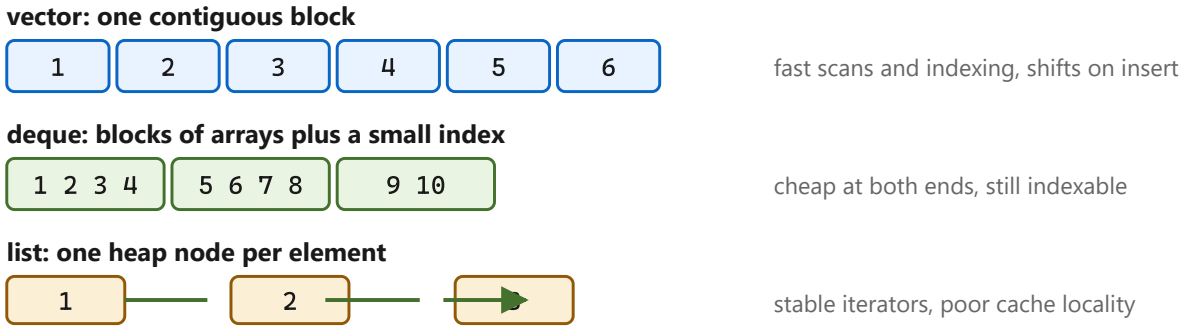
BRACES VS PARENTHESES

```
// parentheses: size 3, every element 7 → {7, 7, 7}
std::vector<int> a(3, 7);
// braces: the initializer_list constructor wins → {3, 7}
std::vector<int> b{3, 7};
```

With braces, a constructor taking `std::initializer_list` is preferred whenever it can match.

---

## Other sequence containers




Memory layout explains the costs: contiguous is cache-friendly, linked is flexible but scattered.

| CONTAINER | STORAGE | BEST AT | WATCH OUT |
|---|---|---|---|
| `std::array<T,N>` | fixed size, inline (stack) | size known at compile time; zero overhead | cannot grow |
| `std::vector<T>` | one contiguous heap block | almost everything; fast iteration and indexing | insert/erase in the middle is O(n) |
| `std::deque<T>` | chunks of arrays | fast push/pop at **both** ends, still indexable | not contiguous |
| `std::list<T>` | doubly linked nodes | O(1) insert/erase anywhere *if you hold an iterator*; iterators stay valid | no indexing, a heap allocation per element, poor cache behaviour |
| `std::forward_list<T>` | singly linked nodes | minimal memory list | forward iteration only |

| OPERATION | VECTOR | DEQUE | LIST |
|---|---|---|---|
| index `[i]` | O(1) | O(1) | not available |
| `push_back` | amortised O(1) | O(1) | O(1) |
| `push_front` | O(n) | O(1) | O(1) |
| insert / erase in the middle | O(n) | O(n) | O(1) at a known position |
| find a value | O(n) | O(n) | O(n) |

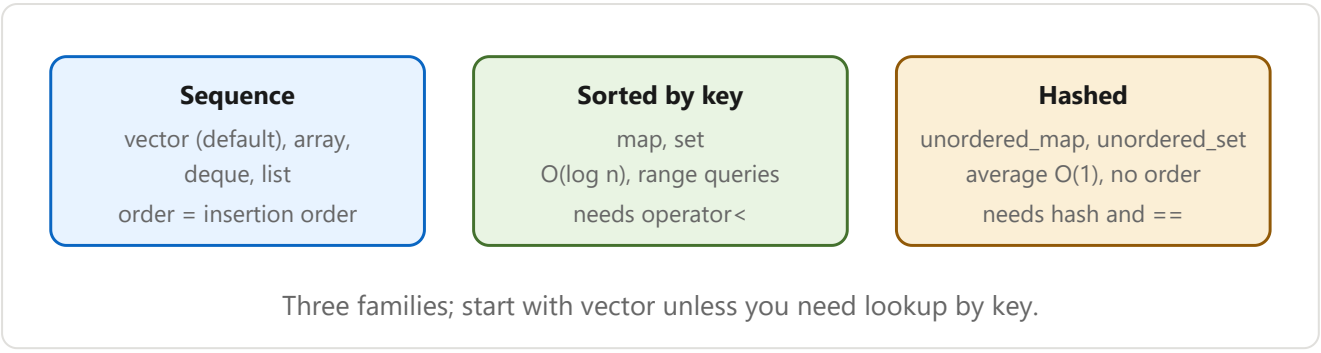
## Container adaptors

They wrap another container and restrict the interface: `std::stack` (LIFO: `push/pop/top`), `std::queue` (FIFO: `push/pop/front`) and `std::priority_queue` (a heap: `top()` is the **largest** element by default).

```
// largest first
std::priority_queue<int> maxHeap;
// smallest first
std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
```

## Which container should I use?

| I NEED TO... | USE | WHY |
|---|---|---|
| iterate over items, access by index, mostly append at the end | `std::vector` | Contiguous memory: fastest iteration and indexing, amortised O(1) `push_back`. This is the default choice. |
| hold a fixed number of items known at compile time | `std::array` | Lives inline (no heap), knows its size, zero overhead over a C array. |
| push and pop at both ends (a deque of work) | `std::deque` | O(1) push/pop at front and back while still allowing indexing. |
| insert/erase often in the middle while holding iterators that must stay valid | `std::list` | O(1) splice/insert/erase at a known position and stable iterators. Costs: a heap allocation per node and poor cache locality. Measure before choosing it. |
| look up values by key; order does not matter | `std::unordered_map` | Hash table: average O(1) find/insert/erase. Needs a hash and `operator==`. |
| look up by key AND iterate in sorted order, or find the nearest key | `std::map` | Balanced tree: O(log n), sorted iteration, `lower_bound` / `upper_bound` for range queries. |
| check membership / remove duplicates (no values attached) | `std::unordered_set` | Hash set: average O(1) contains. Use `std::set` if you also need the elements sorted. |
| count how often each word occurs | `std::unordered_map<std::string, int>` | `counts[word]++` inserts a zero on first sight and increments in one expression. |
| last-in first-out (undo, DFS, matching brackets) | `std::stack` | Adaptor over deque/vector exposing only push, pop and top. |
| first-in first-out (task queue, BFS) | `std::queue` | Adaptor over deque exposing push, pop, front and back. |
| always process the largest (or smallest) item next | `std::priority_queue` | A binary heap: O(log n) push/pop, O(1) top. Largest first by default; pass `std::greater` for smallest first. |




Three families; start with vector unless you need lookup by key.

KEY POINT "vector vs list?" Answer with *memory*: contiguous means cache-friendly, so vector wins even for middle inserts at modest sizes. List wins for stable iterators and cheap splicing.

---

## Common pitfalls

`optional` and `variant` are covered in optional, variant and expected.

| PITFALL | FIX |
|---|---|
| Using an iterator/pointer/reference after a `push_back` reallocated the vector | `reserve` up front, or re-fetch the iterator, or index instead |
| `map[key]` silently inserts | use `find` / `contains` to test; `at()` to read-or-throw |
| `v.size() - 1` on an empty vector (`size()` is unsigned, so it wraps to a huge number) | check `empty()` first or write `i + 1 < v.size()` |
| Passing a big container by value | pass `const std::vector<T>&` |
| `std::vector<bool>` is a packed-bit special case with proxy references | use `std::vector<char>` or `std::bitset` |
| `auto x = v[0]` copies the element | `const auto& x = v[0]` |
| `std::find` on a `map` is O(n) | use the member `m.find(key)`, O(log n) |

## Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

| SECTION | BOOK | PART OF THE BOOK | PDF PAGES |
|---|---|---|---|
| **vector: the workhorse** | C++ Primer | 3.3 Library vector type | 125 to 134 |
| | C++ Primer | 9.3 Sequential container operations | 370 to 383 |
| | C++ Primer | 9.4 How a vector grows | 384 to 388 |
| | Programming: Principles and Practice | 3.5 to 3.6 Functions; vector introduced | 81 to 89 |
| | Programming: Principles and Practice | 17.6 to 17.9 Other operations, remaining Vector problems, changing size | 513 to 523 |
| | Effective Modern C++ | Item 42, Consider emplacement instead of insertion | 310 to 320 |
| | C++ Move Semantics | Chapter 15 Move semantics in standard library types | 229 to 250 |
| **Other sequence containers** | C++ Primer | 9.1 to 9.2 Overview and the container library | 355 to 369 |
| | C++ Primer | 9.6 Container adaptors | 397 to 402 |
| | C++ Primer | 10.6 Container-specific algorithms | 444 to 447 |
| | Programming: Principles and Practice | 19.1 to 19.3, 19.6 Containers, sequences and iterators, linked lists, vector/list/string | 559 to 572 and 585 to 586 |
| **Which container should I use?** | Programming: Principles and Practice | 20.1 to 20.6 map, unordered_map, set, container overview | 591 to 609 |
| **Common pitfalls** | C++ Primer | 9.3 Sequential container operations | 370 to 383 |